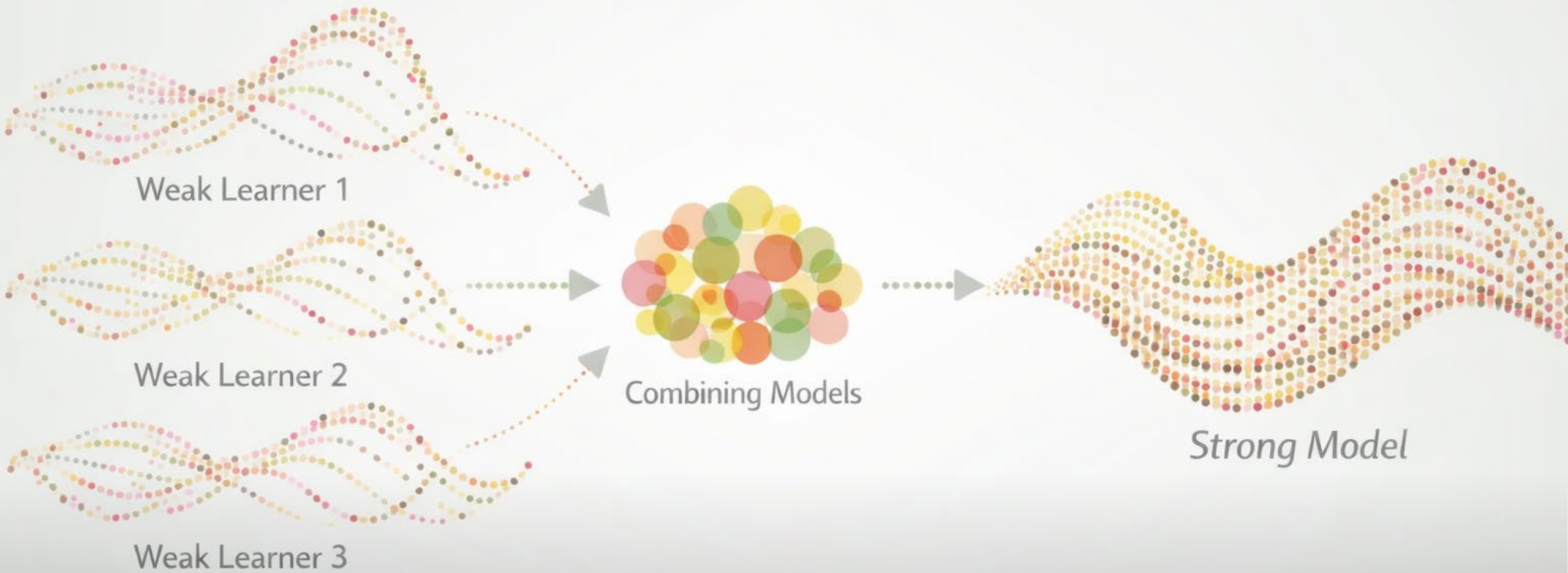




BOOSTING

BOOSTING

- Boosting is a technique that was developed to improve **weak learners**
 - **Models** that are just barely better than random guessing
 - Simple (few tuning parameters) and fast to train
 - Decision “stumps” are commonly used weak learners
 - The general idea of boosting methods is to train models sequentially and try to “*fix the mistakes*” of the preceding model
-

COMMON BOOSTING ALGORITHMS

- Adaptive Boosting (AdaBoost) — increase the weight of misclassified observations
- Gradient Boosting — learn and then correct the mistakes of the previous step
 - More efficient implementations:
 - XGBoost (Extreme gradient boosting)
 - LightGBM (Light gradient boosting machine)

ADABOOST

ADABOOST

- Adaptive Boosting (AdaBoost)
- Introduced in 1997
- The main idea is to “pay more attention” to instances that were misclassified or had large errors in the previous model
- Focus learning on difficult instances
 - “Study” topics you missed last time

ADABOOST

1. Assign each instance a weight, generally $w_i = (1/n)$
(index i refers to the i^{th} observation)

2. Build a weak learner and make predictions
(index j refers to the j^{th} classifier)

Compute error rate of model: $\epsilon_j = \frac{\sum w_i I(y_i \neq \hat{y}_i)}{\sum w_i}$

Compute the weight of the classifier: $\alpha_j = \frac{1}{2} \log \left(\frac{1 - \epsilon_j}{\epsilon_j} \right)$

(α_j will be higher for models with higher accuracy)

(this is the weight of the classifier's vote in the final ensemble prediction)

ADABOOST

5. Update the weights for the instances

$$w_{i,new} = w_{i,old} \cdot \exp(\alpha_j) \text{ if } y_i \neq \hat{y}_i$$
$$w_{i,new} = w_{i,old} \cdot \exp(-\alpha_j) \text{ if } y_i = \hat{y}_i$$

(and normalize by dividing by $\sum w_i$)

6. The next classifier will seek to minimize the weighted error, which now disproportionately penalized errors on higher-weighted instances

- This essentially makes the model focus on learning the instances that were misclassified in the previous step

ADABOOST

7. Update final model prediction

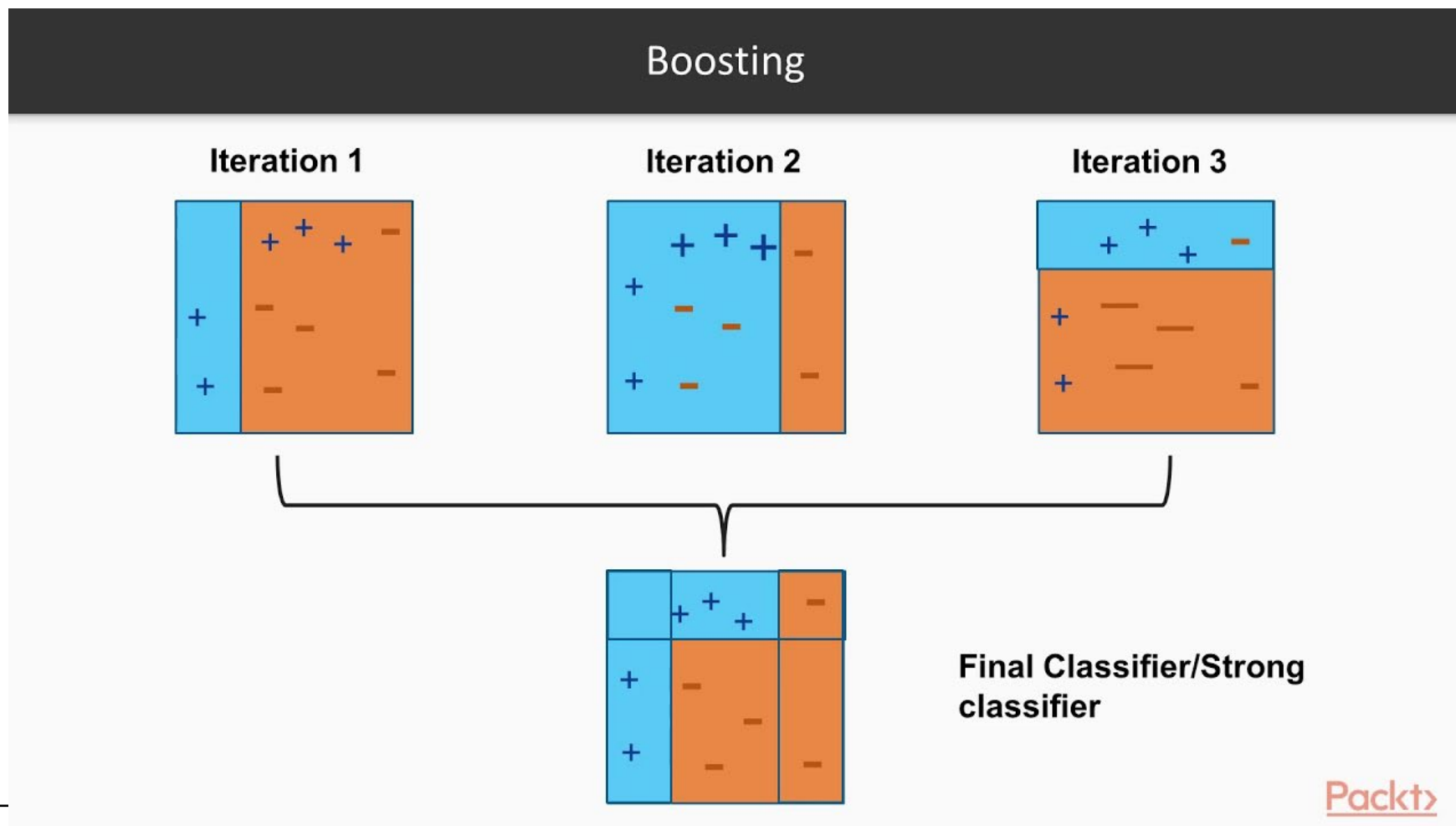
$$F_{new}(x) = F_{old} + \alpha \cdot \hat{y}_{new}$$

8. A learning rate parameters is often used to regularize and make the model training process more conservative

$$F_{new}(x) = F_{old} + \eta \cdot \alpha \cdot \hat{y}_{new}$$

- A low learning rate ($\eta < 1$) means that updates to the final model prediction are smaller
 - This can sometimes reduce overfitting by making the model's adjustments more gradual
-

ADABOOST



GRADIENT BOOSTING

GRADIENT BOOSTING

- Gradient Boosting directly targets the residuals (regression) or gradient of the loss function (classification)
- At each step, a new weak learner is fit to the residual or pseudo-residuals

GRADIENT BOOSTING

1. Train a weak learner and make predictions

2. Calculate the residuals (regression) as:

$$\text{Residual} = \text{Actual Target} - \text{Current Prediction}$$

3. Train a new weak learner that tries to predict the residuals from the previous step

4. Update the Predictions as:

$$\text{New Predictions} = \text{Current Prediction} + \lambda \times (\text{Predictions of New Model})$$

Learning Rate

“Predicted
Residuals”

5. Repeat for a predefined number of iterations or until a stopping criterion is reached

6. Final prediction:

$$F_M(x) = F_0(x) + \sum_{m=1}^M \lambda h_m(x)$$

EXAMPLE (WITH $\lambda = 1$ FOR SIMPLICITY)

Current Target	Current Predictions					
Y	Yhat 1	Resid-1	Yhat-2	Resid-2	Yhat-3	FinalPred iction
0.05	0.12	-0.07	-0.09	0.02	0.01	0.04
0.59	0.53	0.07	0.04	0.03	-0.07	0.5
0.17	0.12	0.04	0.04	0.0	0.01	0.17
-0.07	0.12	-0.19	-0.09	-0.1	0.01	0.04
0.34	0.12	0.22	0.16	0.06	0.01	0.29

EXAMPLE IN A TABLE

Current
Target

Y	Yhat 1	Resid-1	Yhat-2	Resid-2	Yhat-3	FinalPrediction
0.05	0.12	-0.07	-0.09	0.02	0.01	0.04
0.59	0.53	0.07	0.04	0.03	-0.07	0.5
0.17	0.12	0.04	0.04	0.0	0.01	0.17
-0.07	0.12	-0.19	-0.09	-0.1	0.01	0.04
0.34	0.12	0.22	0.16	0.06	0.01	0.29

EXAMPLE IN A TABLE

		Current Target	Current Predictions			
Y	Yhat 1	Resid-1	Yhat-2	Resid-2	Yhat-3	FinalPrediction
0.05	0.12	-0.07	-0.09	0.02	0.01	0.04
0.59	0.53	0.07	0.04	0.03	-0.07	0.5
0.17	0.12	0.04	0.04	0.0	0.01	0.17
-0.07	0.12	-0.19	-0.09	-0.1	0.01	0.04
0.34	0.12	0.22	0.16	0.06	0.01	0.29

EXAMPLE IN A TABLE

Current
Target

Y	Yhat 1	Resid-1	Yhat-2	Resid-2	Yhat-3	FinalPrediction
0.05	0.12	-0.07	-0.09	0.02	0.01	0.04
0.59	0.53	0.07	0.04	0.03	-0.07	0.5
0.17	0.12	0.04	0.04	0.0	0.01	0.17
-0.07	0.12	-0.19	-0.09	-0.1	0.01	0.04
0.34	0.12	0.22	0.16	0.06	0.01	0.29

EXAMPLE IN A TABLE

				Current Target	Current Predictions	
Y	Yhat 1	Resid-1	Yhat-2	Resid-2	Yhat-3	FinalPred iction
0.05	0.12	-0.07	-0.09	0.02	0.01	0.04
0.59	0.53	0.07	0.04	0.03	-0.07	0.5
0.17	0.12	0.04	0.04	0.0	0.01	0.17
-0.07	0.12	-0.19	-0.09	-0.1	0.01	0.04
0.34	0.12	0.22	0.16	0.06	0.01	0.29

EXAMPLE IN A TABLE

Y	Yhat 1	Resid-1	Yhat-2	Resid-2	Yhat-3	Final Prediction
0.05	0.12	-0.07	-0.09	0.02	0.01	0.04
0.59	0.53	0.07	0.04	0.03	-0.07	0.5
0.17	0.12	0.04	0.04	0.0	0.01	0.17
-0.07	0.12	-0.19	-0.09	-0.1	0.01	0.04
0.34	0.12	0.22	0.16	0.06	0.01	0.29

TUNING PARAMETERS

- Learning Rate
 - The learning rate (λ) is a shrinkage parameter that affects the learning by scaling the contribution of each model
 - A smaller learning rate makes smaller adjustments and requires more estimators
 - A higher learning rate allows the model to learn faster but can potentially rapidly converge to a suboptimal solution
 - Could potentially stop improving or get worse with additional learners

TUNING PARAMETERS

- **Number of estimators**

- Number of sequential learners added to the models
- Too few estimators → underfitting (model has not reduced training loss sufficiently)
- Too many estimators → overfitting (model begins fitting noise in the training data)
- Early stopping can help select an appropriate number

- **Learning Rate**

- Controls how much each new tree contributes to the overall model
- Smaller learning rate → requires more trees, but typically improves generalization
- Larger learning rate → requires fewer trees, but increases risk of overfitting or unstable updates

- **Trade-off**

- Lower learning rate + more trees → slower, more stable learning
 - Higher learning rate + fewer trees → faster, more aggressive learning
 - Effective model complexity depends on both parameters together
-

GRADIENT BOOSTING FOR CLASSIFICATION

- The concept of “residuals” is a little different in the classification setting
 - The residual or pseudo residuals are the gradient of the loss function with respect to the predictions.
- For example, in binary classification:
 - The loss function is usually the logistic loss function, also known as log loss or binary cross-entropy loss:

$$L(y, p) = -y\log(p) - (1 - y)\log(1 - p)$$

* y is the actual label (0 or 1)

* p is the predicted probability of the instance being in class 1

GRADIENT BOOSTING FOR CLASSIFICATION

- The gradient of the loss function with respect to p :

$$\frac{\partial L}{\partial p} = \frac{p - y}{p(1 - p)}$$

- However, this is generally simplified by working with the log odds,

$$z, \text{ where } p = \frac{1}{1 + e^{-z}}$$

- The gradient with respect to the log odds turns out to be:

$$\frac{\partial L}{\partial z} = y - p$$

IMPLEMENTATIONS OF GRADIENT BOOSTING

- **Scikit-Learn**

- GradientBoostingClassifier / GradientBoostingRegressor
- Easy to use and integrates well with sklearn pipelines
- Can be slower on large datasets
- HistGradientBoosting provides a faster, histogram-based implementation

- **XGBoost (Extreme Gradient Boosting)**

- Highly optimized and efficient
- Supports L1 and L2 regularization
- Handles missing values automatically
- Parallelized and widely used in industry

- **LightGBM (Light Gradient Boosting Machine)**

- Very fast for large datasets
- Histogram-based learning for efficiency
- Leaf-wise tree growth strategy
- Native handling of categorical features
- Memory efficient

- **CatBoost**

- Strong performance with minimal tuning
 - Excellent handling of categorical variables
 - Reduces preprocessing requirements
-

INSTALLING BOOSTING LIBRARIES

- **XGBoost**
 - Using pip:
pip install xgboost
 - Using conda:
conda install -c conda-forge py-xgboost
 - **LightGBM**
 - Using pip:
pip install lightgbm
 - Using conda:
conda install -c conda-forge lightgbm
 - **CatBoost**
 - Using pip:
pip install catboost
 - Using conda:
conda install -c conda-forge catboost
-